МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение высшего образования «Санкт-Петербургский политехнический университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 Математика и компьютерные науки

Методы тестирования программного обеспечения

Отчет по лабораторной работе №2 «Тестирование методом черного и белого ящика»

Студент
группы 5130201/30101 _____ Шишмаков В.О.

Преподаватель _____ Курочкин М. А.

«____»_____ 2026 г.

Санкт-Петербург, 2026

Содержание

Введение **3**

1 Постановка задачи **4**

2 Описание методов тестирования **5**

2.1 Метод черного ящика 5

2.1.1 Эквивалентное разбиение 5

2.1.2 Анализ граничных значений 6

2.1.3 Причинно-следственные диаграммы 6

2.2 Метод белого ящика 9

2.2.1 Покрытие операторов 9

2.2.2 Покрытие решений 10

2.2.3 Покрытие условий 10

2.2.4 Покрытие решений и условий 10

2.2.5 Комбинаторное покрытие условий 10

3 Тестирование программы №1 **11**

3.1 Описание программы 11

3.2 Тестирование методом «черного» ящика 12

3.2.1 Разбиение на классы эквивалентности 12

3.2.2 Анализ граничных условий 13

3.2.3 Причинно-следственная диаграмма 14

3.2.4 Результаты тестирования программы методом чёрного ящика 16

4 Тестирование программы №2 **18**

4.1 Описание программы 18

4.2 Тестирование методом белого ящика 20

4.2.1 Покрытие операторов 21

4.2.2 Покрытие решений 21

4.2.3 Покрытие условий 22

4.2.4 Покрытие решений и условий 22

4.2.5 Комбинаторное покрытие условий 23

4.2.6 Результаты тестирования методом белого ящика 24

5 Заключение **26**

Введение

Одним из важнейших этапов в процессе создания современного программного обеспечения является тестирование. Выделяют различные виды тестирования: модульное, интеграционное, функциональное, системное и приемочное. В рамках данной работы основное внимание уделяется модульному тестированию.

Модульное тестирование представляет собой базовый вид проверки, с которого, как правило, начинается оценка корректности функционирования программы. Оно предполагает проверку отдельных компонентов программного продукта: классов, модулей и функций. Основная цель модульного тестирования — обнаружение дефектов, то есть случаев, когда реальное поведение модуля расходится с его спецификацией.

В практике модульного тестирования применяются два основных подхода: метод «черного ящика» и метод «белого ящика».

Оба этих подхода будут рассмотрены в данной работе.

1 Постановка задачи

Цель: провести тестирование двух программ методами черного и белого ящика.
Задачи:

1. Ознакомиться с подходами к модульному тестированию, а именно с методами черного и белого ящика.
2. Составить тесты для двух программ и провести их.
3. Провести анализ результатов тестирования.

2 Описание методов тестирования

2.1 Метод черного ящика

Одним из ключевых подходов к тестированию является метод «черного ящика», известный также как тестирование, ориентированное на данные, либо тестирование на основе входных и выходных параметров. Согласно данному подходу, программа рассматривается как «черный ящик», внутренности и логика которого остаются скрыты и не принимаются во внимание. Основное внимание уделяется выявлению ситуаций, в которых реальная работа программы отклоняется от требований спецификации.

В рамках данного подхода необходимо выполнить проверку программы, используя такие техники, как разбиение входных данных на классы эквивалентности, анализ граничных значений и построение причинно-следственных диаграмм.

Далее будут подробно описаны три указанных приема тестирования методом «черного ящика».

2.1.1 Эквивалентное разбиение

Качественно разработанный тест должен охватывать значительную часть прочих потенциальных тестов, то есть предоставлять информацию о возможном присутствии или отсутствии дефектов в сценариях, не охваченных данным конкретным набором входных данных. В связи с этим целесообразно разделить все множество входных данных программы на ограниченное количество классов эквивалентности. Это позволит с достаточной уверенностью считать, что проверка на одном показательном значении из определенного класса равносильна проверке на любом другом значении из того же класса.

Процесс выделения классов эквивалентности заключается в последовательном анализе каждого входного условия и его разделении на две или более групп. Различают два типа классов эквивалентности: корректные классы, которые включают допустимые входные значения программы, и некорректные классы, представляющие все остальные возможные состояния.

Определение тестов по построенным классам эквивалентности состоит из трех этапов:

1. Присвоить каждому классу эквивалентности номер.
2. Разработать тесты, которые одновременно покрывают максимально возможное количество еще не задействованных корректных классов,

и продолжать этот процесс до тех пор, пока все корректные классы не будут охвачены.

3. Создать тесты, каждый из которых покрывает ровно один из оставшихся некорректных классов, и повторять это до полного охвата всех некорректных классов.

2.1.2 Анализ граничных значений

Граничные условия представляют собой ситуации, которые возникают в окрестностях предельных значений, определяющих классы эквивалентности входных и выходных данных. Метод анализа граничных значений имеет два ключевых отличия от подхода, основанного на разбиении на классы эквивалентности.

1. Вместо произвольного выбора любого значения из класса эквивалентности в качестве представителя всего класса, анализ граничных значений требует выбора таких элементов, которые позволяют проверить именно каждую границу класса.

2. При построении тестовых сценариев внимание уделяется не только входным данным (то есть области допустимых входных значений), но и выходным данным (классам эквивалентности результатов).

2.1.3 Причинно-следственные диаграммы

Причинно-следственная диаграмма представляет собой формализованный графический способ описания спецификации, изначально сформулированной на естественном языке. Базовая нотация таких диаграмм показана на рисунке 1. Каждый узел диаграммы может принимать одно из двух значений: 0 или 1, где 0 означает «отсутствует» или «ложно», а 1 — «присутствует» или «истинно».

Данный метод призван восполнить недостаток подходов, основанных на анализе граничных значений и разбиении на классы эквивалентности, которые не учитывают комбинации входных условий.

Процесс построения тестов с использованием причинно-следственных диаграмм включает несколько шагов.

1. Спецификация разделяется на фрагменты, более удобные для последующего анализа.

2. В спецификации выделяются причины и следствия.

   Причина — это отдельное входное условие или класс эквивалентности входных данных.

Следствие — это выходное условие либо изменение состояния системы (то есть долгосрочный эффект, который входное воздействие оказывает на программу или систему).

Выявление причин и следствий осуществляется путем тщательного изучения спецификации с подчеркиванием слов и фраз, соответствующих этим понятиям. Каждой причине и каждому следствию присваивается уникальный идентификатор.

3. Смысловое содержание спецификации анализируется и преобразуется в булев граф, отражающий связи между причинами и следствиями. Полученный граф и есть причинно-следственная диаграмма.

4. На диаграмму наносятся ограничения, которые описывают комбинации причин и/или следствий, невозможные в силу синтаксических или внешних по отношению к программе условий.

   - Ограничение E (исключающее) предусматривает, что из двух величин a и b только одна может принимать значение 1 (одновременное равенство единице запрещено, однако допустима ситуация, когда оба равны 0).
   - Ограничение I (включающее) требует, чтобы хотя бы одна из величин a, b или c имела значение 1 (то есть запрещён случай, когда все три одновременно равны 0).
   - Ограничение O (ровно одно) означает, что ровно одна из двух величин a или b должна быть равна 1.
   - Ограничение R (требует) устанавливает, что a может быть равно 1 только в том случае, если b также равно 1 (иначе говоря, при $b = 0$ значение a не может быть 1). На рисунке 2 приведены условные обозначения для этих ограничений.

5. Посредством последовательного анализа состояний условий диаграмма преобразуется в ограниченную таблицу решений. Каждый столбец этой таблицы соответствует отдельному тестовому сценарию.

6. Столбцы полученной таблицы решений служат основой для формирования конкретных тестов.

Базовая нотация причинно-следственных диаграмм представлена на рисунке 1.

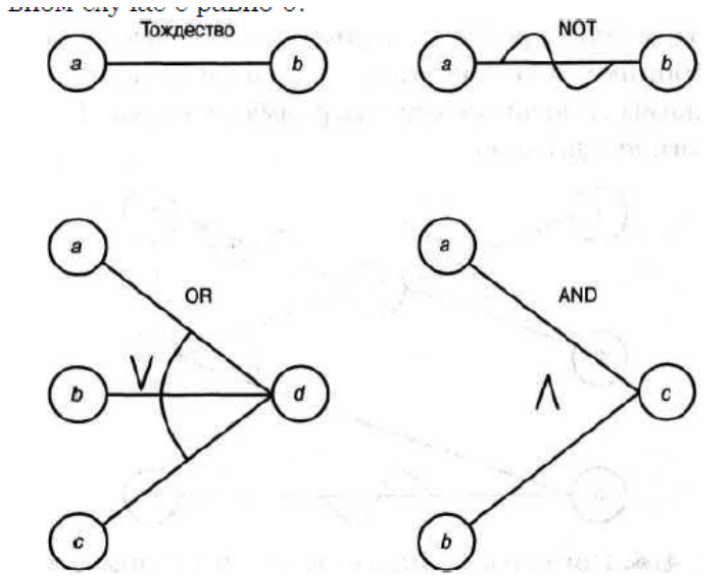

Рис. 1: Базовая нотация для причинно-следственных диаграмм

Процедура заполнения очередного столбца таблицы решений выполняется следующим образом.

1. Выбрать следствие, которое должно находиться в состоянии 1.
2. Двигаясь по связям диаграммы от выбранного узла в обратном направлении, определить комбинации причин (с учетом наложенных ограничений), которые приводят к установке этого следствия в 1.
   - Если при обратном проходе встречается элемент OR, на выходе которого требуется получить 1, то не следует одновременно устанавливать в 1 более одного входа этого элемента.
   - Если при обратном проходе встречается элемент AND, на выходе которого требуется получить 0, то необходимо перечислить все возможные комбинации входов, дающие на выходе 0. Однако при анализе ситуации, где один из входов равен 0, а один или несколько других входов равны 1, нет необходимости перечислять все варианты состояний остальных входов, равных 1.
   - Если при обратном проходе встречается элемент AND, на выходе которого требуется получить 0, то достаточно указать только одно условие, при котором все входы являются нулями.

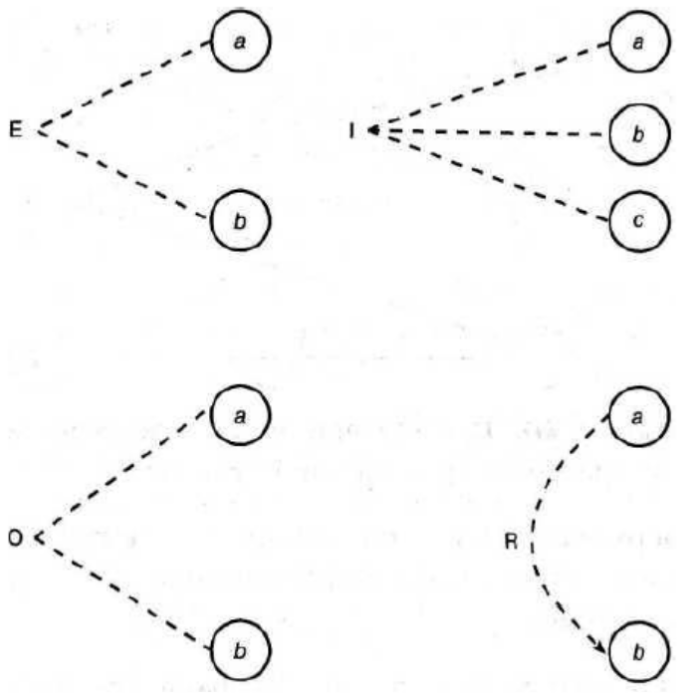

Рис. 2: Базовая нотация ограничений для причинно-следственных диаграмм

3. Для каждой выявленной комбинации причин формируется отдельный столбец в таблице решений.

4. Для каждой такой комбинации определяются состояния всех остальных следствий, которые затем заносятся в соответствующий столбец таблицы.

2.2 Метод белого ящика

В рамках подхода, известного как тестирование методом «белого ящика», предоставляется возможность изучать внутреннее устройство программы. Основываясь на этом принципе, специалист по тестированию выбирает входные данные, анализируя логическую структуру программы.

Необходимо выполнить проверку программы, используя описанные ниже методы.

2.2.1 Покрытие операторов

Минимальным требованием к тестированию методом «белого ящика» является выполнение каждого оператора программы как минимум однократно. Однако, хотя это условие и необходимо, его недостаточно для обеспечения полноценной проверки.

2.2.2 Покрытие решений

Согласно данному критерию, необходимо подобрать такой набор тестов, при котором каждое логическое условие в программе хотя бы один раз принимает значение «истина» и хотя бы один раз — «ложь». Другими словами, каждая возможная ветвь для всех инструкций, реализующих ветвление, должна быть пройдена минимум однократно.

2.2.3 Покрытие условий

Этот критерий требует, чтобы набор тестов обеспечивал для каждого элементарного условия, входящего в состав составного условия в точке ветвления, выполнение как минимум один раз значения «истина» и один раз значения «ложь».

2.2.4 Покрытие решений и условий

В соответствии с данным критерием тестовый набор считается достаточным, если выполнены следующие условия: каждое элементарное условие внутри составного решения принимает оба возможных логических значения хотя бы однажды; каждый возможный результат самого решения также достигается минимум один раз; и каждая точка входа в программу или модуль активируется по крайней мере однократно.

2.2.5 Комбинаторное покрытие условий

Данный критерий предполагает формирование такого набора тестов, который обеспечивает проверку всех возможных комбинаций значений элементарных условий в каждом логическом решении, а также активацию каждой точки входа в программу как минимум однократно.

3 Тестирование программы №1

3.1 Описание программы

Автор: Митяев Олег.
Название: Проверка простоты числа.
Дано:

- n — целое число.

Требуется: определить, является ли число n простым.
Ограничения:

1. n вводится как целое число.
2. При некорректном вводе выводится сообщение `ошибка ввода` и выполняется повтор ввода.
3. Числа $n \leq 1$ считаются непростыми.
4. Проверка простоты выполняется перебором делителей до $[\sqrt{n}]$.

Спецификация:

Входные данные	**Ожидаемый результат**	**Статус**	**Комментарий**
7	вывод: простое	Корректно	Обычный случай (простое число).
4	вывод: не простое	Корректно	Обычный случай (составное число).
2	вывод: простое	Корректно	Граница: минимальное простое.
1	вывод: не простое	Корректно	Граница: $n \leq 1$ непростое.
-5	вывод: не простое	Корректно	Отрицательные числа непростые.
abc	вывод: ошибка ввода; ожидание повторного ввода	Ошибка ввода	Проверка обработки некорректного ввода (строка вместо числа).
80 12	вывод: ошибка ввода; ожидание повторного ввода	Ошибка ввода	Обработка некорректного ввода (число разделено пробелом).
3.45	вывод: ошибка ввода; ожидание повторного ввода	Ошибка ввода	Обработка некорректного ввода (не целое число).

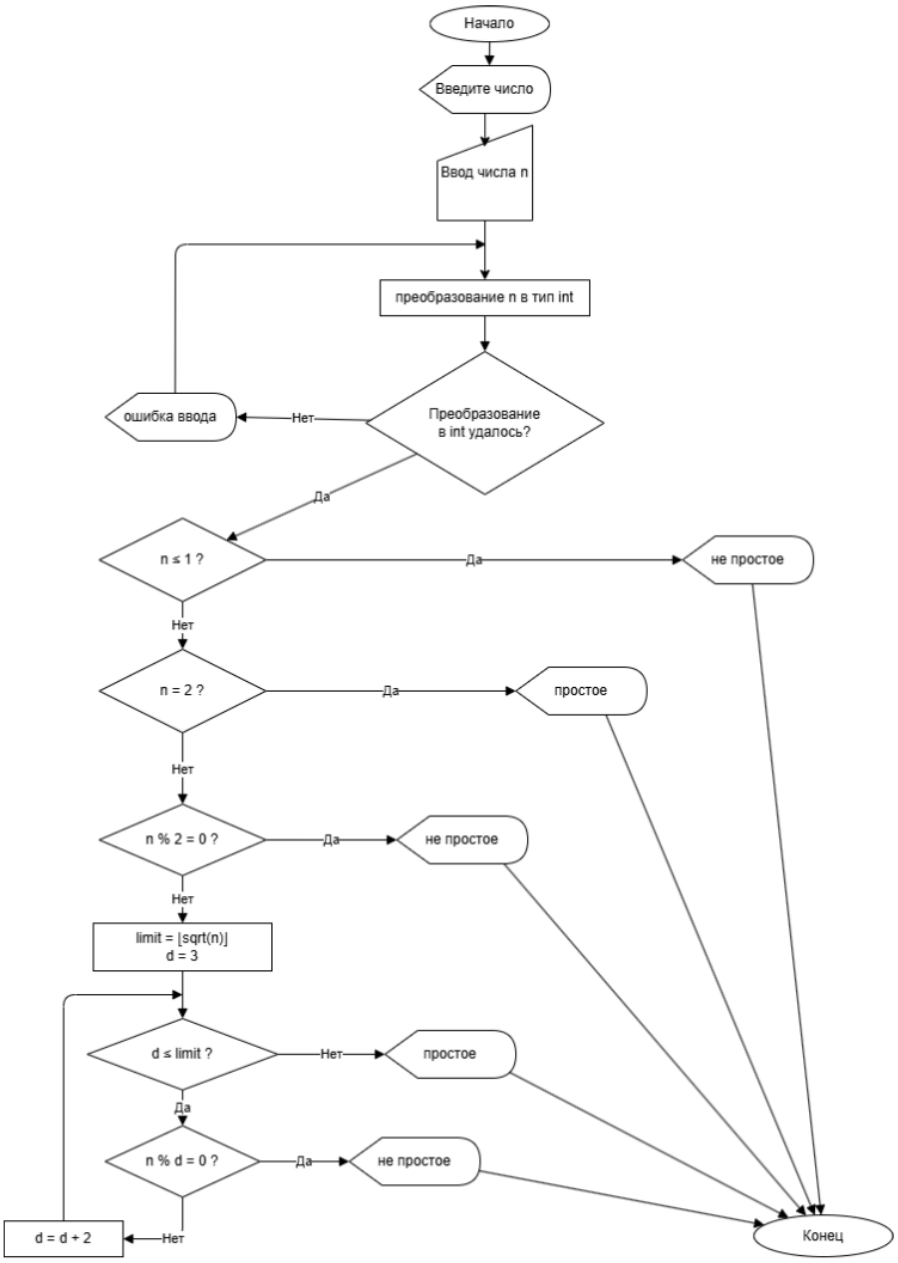

Рис. 3: Блок-схема

3.2 Тестирование методом «черного» ящика

3.2.1 Разбиение на классы эквивалентности

Составлено разбиение на классы эквивалентности, исходя из ограничений для тестируемой программы.

Таблица 1: Классы эквивалентности для n

Входное условие	Допустимые классы	Недопустимые классы
Целое число n	Отрицательное число (1)	Нецелое вещественное число (8)

	Ноль (2)	Строка (буквы) (9)
	Единица (3)	Несколько чисел через пробел (10)
	Минимальное простое (4)	Пустая строка (11)
	Четное число > 2 (5)	Спецсимволы / смешанный ввод (12)
	Нечетное простое число (6)	
	Нечетное составное число (7)	

В таблице 2 представлены тесты, покрывающие все классы эквивалентности.

Таблица 2: Тесты, покрывающие классы эквивалентности

N	n	Класс эквивалентности	Ожидаемый результат	Фактический результат	Результат
1	-5	(1)	Число не простое	Число не простое	Пройден
2	0	(2)	Число не простое	Число не простое	Пройден
3	1	(3)	Число не простое	Число не простое	Пройден
4	2	(4)	Число простое	Число простое	Пройден
5	8	(5)	Число не простое	Число не простое	Пройден
6	17	(6)	Число простое	Число простое	Пройден
7	15	(7)	Число не простое	Число не простое	Пройден
8	7.5	(8)	Ошибка ввода; повторный ввод	Ошибка ввода; повторный ввод	Пройден
9	abc	(9)	Ошибка ввода; повторный ввод	Ошибка ввода; повторный ввод	Пройден
10	12 5	(10)	Ошибка ввода; повторный ввод	Ошибка ввода; повторный ввод	Пройден
11	«»	(11)	Ошибка ввода; повторный ввод	Ошибка ввода; повторный ввод	Пройден
12	10a?	(12)	Ошибка ввода; повторный ввод	Ошибка ввода; повторный ввод	Пройден

Все 12 тестов прошли успешно.

3.2.2 Анализ граничных условий

В данной программе можно выделить следующие граничные значения с точки зрения алгоритма:

1. Переход от наибольшего непростого к наименьшему простому (1, 2).
2. Переход между наименьшим четным составным (3, 4, 5).
3. Делитель ровно на границе: $\sqrt{9} = 3$ (8, 9, 10).

Для каждой из границ составлены тесты, соответствующие граничащим значениям.

Таблица 3: Проверка граничных условий

N	n	Ожидаемый результат	Фактический результат	Результат
1	1	Число не простое	Число не простое	Пройден
1	2	Число простое	Число простое	Пройден
3	3	Число простое	Число простое	Пройден
4	4	Число не простое	Число не простое	Пройден
5	5	Число простое	Число простое	Пройден
6	8	Число не простое	Число не простое	Пройден
7	9	Число не простое	Число не простое	Пройден
8	10	Число не простое	Число не простое	Пройден

При проведении тестирования программа успешно обработала все граничные условия.

3.2.3 Причинно-следственная диаграмма

Для построения причинно-следственной диаграммы введём следующие обозначения:

Причины:

1. Ввод является целым числом.
2. Число $n \leq 1$.
3. Число $n = 2$.
4. Число n чётное и $n \neq 2$.
5. Среди нечётных чисел от 3 до $\lfloor \sqrt{n} \rfloor$ найден делитель n.

Следствия:

21. Программа выводит сообщение об ошибке ввода и запрашивает повторный ввод.
22. Программа выводит результат: `не простое` (по условию $n \leq 1$).
23. Программа выводит результат: `простое` (граничный случай $n = 2$).
24. Программа выводит результат: `не простое` (чётное число $n \neq 2$).
25. Программа выводит результат: `не простое` (найден делитель при переборе).
26. Программа выводит результат: `простое` (делитель не найден).

Промежуточные узлы:

11. Причина 1 истинна И причина 2 ложна (корректный ввод, $n > 1$ — число допущено к дальнейшему анализу).

12. Узел 11 истинен И причина 3 ложна И причина 4 ложна (число нечётное и больше 2 — выполняется перебор делителей).

Логические связи:

1. Следствие 21 (ошибка ввода) истинно, если причина 1 ложна (введено не целое число).
2. Следствие 22 (`не простое`) истинно, если узел 11 ложен И причина 2 истинна (корректный ввод, $n \leq 1$).
3. Следствие 23 (`простое`) истинно, если узел 11 истинен И причина 3 истинна ($n = 2$).
4. Следствие 24 (`не простое`) истинно, если узел 11 истинен И причина 4 истинна (чётное $n \neq 2$).
5. Следствие 25 (`не простое`) истинно, если узел 12 истинен И причина 5 истинна (при переборе найден делитель).
6. Следствие 26 (`простое`) истинно, если узел 12 истинен И причина 5 ложна (делитель при переборе не найден).
7. Узел 11 является конъюнкцией (AND) причин 1 и ¬2.
8. Узел 12 является конъюнкцией (AND) узла 11, ¬3 и ¬4.

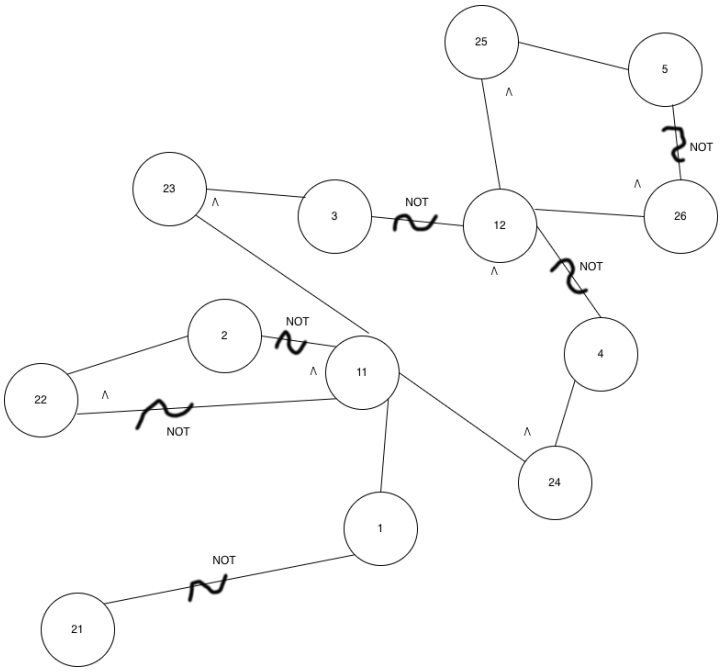

Рис. 4: Причинно-следственная диаграмма

Таблица 4: Таблица решений

	1	**2**	**3**	**4**	**5**	**6**	**7**
Причина 1 (целое число)	0	1	1	1	1	1	1
Причина 2 ($n \leq 1$)	–	1	0	0	0	0	0
Причина 3 ($n = 2$)	–	–	–	1	0	0	0
Причина 4 (n чётное, $\neq 2$)	–	–	–	–	1	0	0
Причина 5 (делитель найден)	–	–	–	–	–	1	0
Узел 11 ($C_1 \wedge \neg C_2$)	0	0	1	1	1	1	1
Узел 12 ($U_{11} \wedge \neg C_3 \wedge \neg C_4$)	0	0	0	0	0	1	1
Следствие 21 (ошибка ввода)	1	0	0	0	0	0	0
Следствие 22 (`не простое`, $n \leq 1$)	0	1	0	0	0	0	0
Следствие 23 (`простое`, $n = 2$)	0	0	0	1	0	0	0
Следствие 24 (`не простое`, чётное)	0	0	0	0	1	0	0
Следствие 25 (`не простое`, делитель найден)	0	0	0	0	0	1	0
Следствие 26 (`простое`, делитель не найден)	0	0	0	0	0	0	1

Таблица 5: Тесты на основе таблицы решений

N	**Комбинация**	**n**	**Ожидаемый рез.**	**Фактич. рез.**	**Результат**
1	0	`abc`	Следствие 21: ошибка ввода; повторный ввод	Следствие 21: ошибка ввода; повторный ввод	Пройден
2	1, 1	`-3`	Следствие 22: `не простое` ($n \leq 1$)	Следствие 22: `не простое`	Пройден
3	1, 0, 1	`2`	Следствие 23: `простое` ($n = 2$)	Следствие 23: `простое`	Пройден
4	1, 0, 0, 1	`8`	Следствие 24: `не простое` (чётное $n \neq 2$)	Следствие 24: `не простое`	Пройден
5	1, 0, 0, 0, 1	`15`	Следствие 25: `не простое` (делитель найден)	Следствие 25: `не простое`	Пройден
6	1, 0, 0, 0, 0	`17`	Следствие 26: `простое` (делитель не найден)	Следствие 26: `простое`	Пройден

3.2.4 Результаты тестирования программы методом чёрного ящика

В результате тестирования методом чёрного ящика было составлено 30 тестов (12 классов эквивалентности + 12 граничных значений + 6 тестов на основе таблицы решений). Все 30 тестов были успешно пройдены, дефектов в программе обнаружено не было.

Тестирование по классам эквивалентности охватило все выделенные

классы входных данных: некорректный ввод (не целое число, строка, несколько чисел, пустая строка), числа $n \leq 0$, граничный случай $n = 1$, минимальное простое $n = 2$, чётные составные числа, нечётные простые и нечётные составные числа. Во всех случаях программа выдала результат, соответствующий спецификации.

Анализ граничных условий подтвердил корректность реализации критических переходов. В частности, граница $n = 1$ / $n = 2$ обработана верно: число 1 определяется как непростое, число 2 — как простое. Наиболее значимой оказалась проверка границы перебора делителей $\lfloor\sqrt{n}\rfloor$: числа $9 = 3^2$ и $25 = 5^2$, чьи делители лежат ровно на верхней границе цикла, корректно определены как непростые, что свидетельствует о правильном использовании оператора $\leq$ в условии завершения цикла. Ошибка типа $<$ вместо $\leq$ привела бы к ложному результату `простое` для таких чисел.

Тестирование на основе таблицы решений позволило верифицировать каждую ветвь алгоритма в отдельности: все шесть столбцов таблицы решений покрыты тестами, каждый из которых активировал ровно одно следствие и дал ожидаемый результат.

Таким образом, все тесты пройдены успешно, что подтверждает корректность основной логики определения простоты числа, правильность обработки граничных случаев и соответствие программы требованиям спецификации.

4 Тестирование программы №2

4.1 Описание программы

Автор: Попов Илья.
Название: алгоритм подсчета вхождений подстроки в массиве строк.
Дано:

1. `arr` — массив строк.
2. `substr` — подстрока для поиска в массиве.

Требуется: найти количество вхождений `count` подстроки в массиве строк.
Ограничения:

1. Длина n подстроки `substr`: $n \in [0; 1000]$
2. Длина n строки `str` в массиве `arr`: $n \in [1; 1000]$
3. Размер массива $n \in [1; 100]$
4. `count` — целое неотрицательное число.

Спецификация:

Входные данные	**Выходные данные**	**Реакция программы**
arr = {"hello "world "hello"} substr = "he"	count = 2	Подстрока встречается дважды. Возвращается значение. Программа завершает работу.
arr = {"hello "world "hello"} substr =	Error: "Подстрока не может быть пустой"	Программа генерирует исключение. Завершение работы программы.
arr = {"hello "world "hello"} substr = "eee"	count = 0	Подстрока не встречается. Возвращается значение. Программа завершает работу.
arr = {} substr = "he"	Error: "Массив пустой"	Программа генерирует исключение. Завершение работы программы.
длина arr > 100 substr = "e"	Error: "Массив слишком большой"	Программа генерирует исключение. Завершение работы программы.
длина элемента в arr > 1000 substr = "e"	Error: "Слишком длинная строка в массиве"	Программа генерирует исключение. Завершение работы программы.

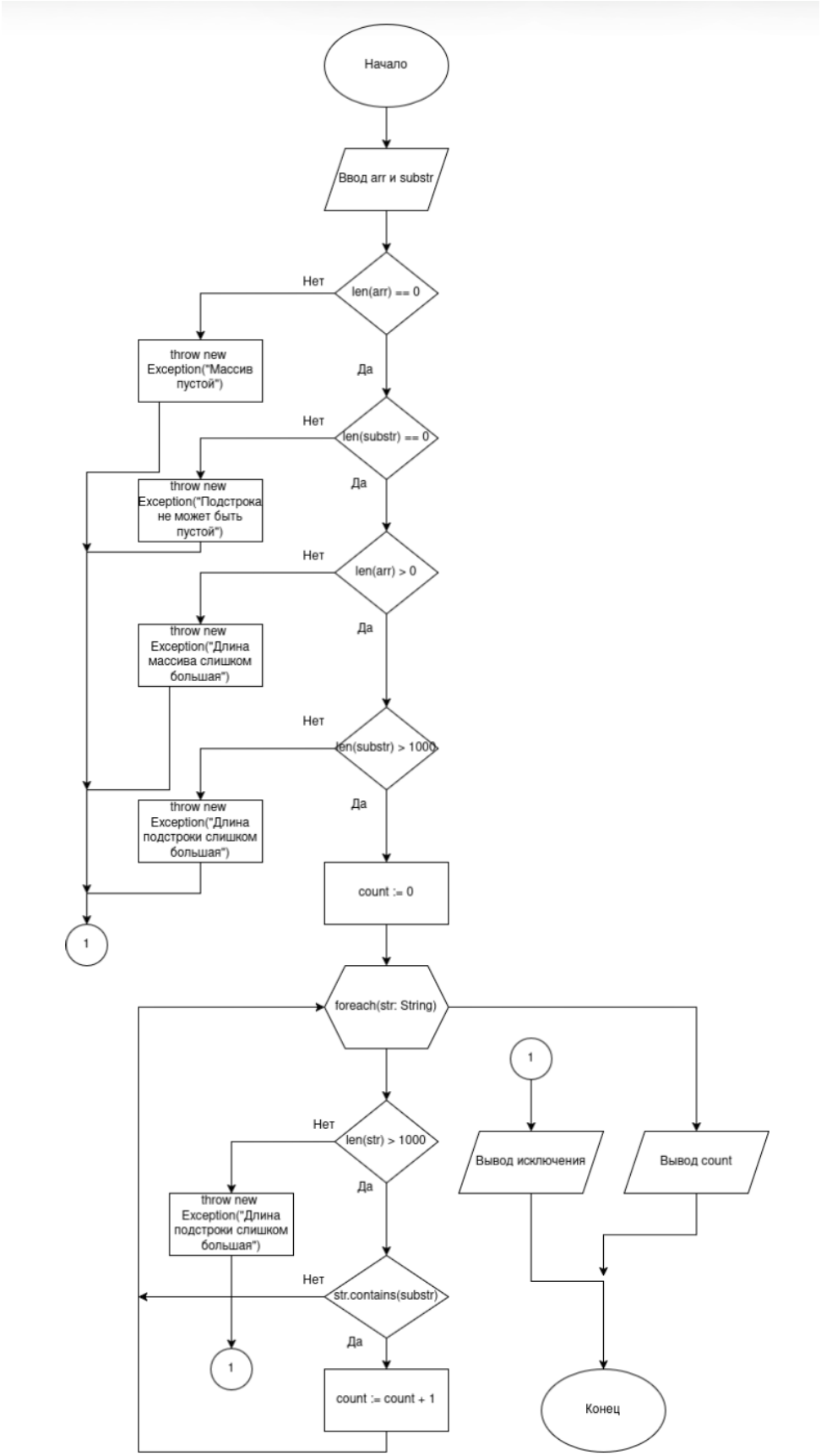

Рис. 5: Блок-схема

4.2 Тестирование методом белого ящика

Алгоритм составления тестов методом «белого» ящика предполагает обход всех возможных путей в теле программы и проверку выполнения каждого оператора не менее одного раза.

В программе можно выделить следующие ключевые условия и соответствующие им ветви:

1. Условие C1: (`len(arr) == 0`):

   Ветвь T (True): Выход из программы с ошибкой «Массив пустой».

   Ветвь F (False): Продолжение выполнения.

2. Условие C2: (`len(substr) == 0`):

   Ветвь T (True): Выход из программы с ошибкой «Подстрока не может быть пустой».

   Ветвь F (False): Продолжение выполнения.

3. Условие C3: (`len(arr) > 100`) (проверка размера массива):

   Ветвь T (True): Выход из программы с ошибкой «Длина массива слишком большая».

   Ветвь F (False): Продолжение выполнения.

4. Условие C4: (`len(substr) > 1000`) (проверка длины подстроки):

   Ветвь T (True): Выход из программы с ошибкой «Длина подстроки слишком большая».

   Ветвь F (False): Продолжение выполнения.

5. Условие C5: (`len(str) > 1000`) (внутри цикла `foreach`):

   Ветвь T (True): Выход из программы с ошибкой «Длина строки слишком большая».

   Ветвь F (False): Продолжение проверки или выход из цикла.

6. Условие C6: (`str.contains(substr)`):

   Ветвь T (True): Увеличение счётчика `count`.

   Ветвь F (False): Переход к следующему элементу.

Возможные пути выполнения:

1. Пустой массив: C1(T) $\rightarrow$ Завершение.
2. Пустая подстрока: C1(F) $\rightarrow$ C2(T) $\rightarrow$ Завершение.
3. Массив слишком большой: C1(F) $\rightarrow$ C2(F) $\rightarrow$ C3(T) $\rightarrow$ Завершение.

4. Подстрока слишком длинная: C1(F) → C2(F) → C3(F) → C4(T) → Завершение.
5. Строка в массиве слишком длинная: C1(F) → ... → C4(F) → цикл → C5(T) → Завершение.
6. Успешный поиск (есть совпадения): C1(F) → ... → C4(F) → цикл → C5(F) → C6(T) → ... → выход из цикла → Вывод `count`.
7. Поиск без совпадений: C1(F) → ... → C4(F) → цикл → C5(F) → C6(F) → ... → выход из цикла → Вывод `count = 0`.
8. Смешанный поиск: C1(F) → ... → C4(F) → цикл с многократным выполнением C5(F), C6(T/F) → выход из цикла → Вывод `count`.

4.2.1 Покрытие операторов

Критерием покрытия является выполнение каждого оператора программы хотя бы один раз. Это необходимое, но не достаточное условие для приемлемого тестирования по принципу белого ящика.

Для покрытия всех операторов достаточно одного теста, который пройдёт по основному пути и не вызовет досрочных завершений.

Таблица 6: Покрытие операторов

N	**arr**	**substr**	**Ожид. результат**	**Факт. результат**	**Путь**	**Статус**
1	{"hello "world "hello"}	"he"	2	2	6	Пройден

Тест №1 выполнил все операторы программы.

4.2.2 Покрытие решений

В соответствии с этим критерием необходимо составить такой набор тестов, при котором каждое условие в программе примет как истинное, так и ложное значения. Таким образом, к тестам, составленным для метода покрытия операторов, необходимо добавить тесты, которые будут проверять все возможные переходы.

Тесты, покрывающие все решения программы, представлены ниже.

Тест №1: Проверяет C1=F, C2=F, C3=F, C4=F, C5=F, C6=T.

Тест №2: Проверяет C6=F, а также выход из цикла без совпадений.

Тест №3: Проверяет C1=T.

Тест №4: Проверяет C2=T.

Тест №5: Проверяет C3=T.

Тест №6: Проверяет C4=T.

Тест №7: Проверяет C5=T.

Все условия были протестированы на T и F.

Таблица 7: Покрытие решений

N	arr	substr	Ожид. рез.	Факт. рез.	C1	C2	C3	C4	C5	C6	Статус
1	{"hello "world"hello"}	"hll"	2	2	F	F	F	F	F	T	Пройден
2	{"cat "dog"}"xyz"		0	0	F	F	F	F	F	F	Пройден
3	{}	"he"	Ошибка: массив пустой	Ошибка: массив пустой	T	–	–	–	–	–	Пройден
4	{"hello"}		Ошибка: пустая подстрока	Ошибка: пустая подстрока	F	T	–	–	–	–	Пройден
5	arr из 101 эл.	"a"	Ошибка: массив слишком большой	Ошибка: массив слишком большой	F	F	T	–	–	–	Пройден
6	{"hello"}	substr дл. 1001	Ошибка: подстрока слишком длинная	Ошибка: подстрока слишком длинная	F	F	F	T	–	–	Пройден
7	{"hello str дл. 1001}	"he"	Ошибка: строка слишком длинная	Ошибка: строка слишком длинная	F	F	F	F	T	–	Пройден

4.2.3 Покрытие условий

В соответствии с этим критерием количество тестов должно быть таким, чтобы все возможные результаты каждого условия в решении выполнялись по крайней мере один раз.

Таблица 8: Покрытие условий

N	arr	substr	Ожид. рез.	Факт. рез.	C1	C2	Статус
1	{"hello "world"}	"he"	1	1	F	F	Пройден
2	{}	"he"	Ошибка: массив пустой	Ошибка: массив пустой	T	–	Пройден
3	{"hello"}		Ошибка: пустая подстрока	Ошибка: пустая подстрока	F	T	Пройден

Тест №1: C1=F, C2=F.
Тест №2: C1=T.
Тест №3: C2=T.
Все элементарные условия были проверены на T и F.

4.2.4 Покрытие решений и условий

Согласно этому критерию набор тестов является достаточно полным, если удовлетворяются следующие требования: каждое условие в решении принимает каждое возможное значение по крайней мере один раз, каждый возможный исход решения проверяется по крайней мере один раз и

каждой точке входа управление передаётся по крайней мере один раз.

Тесты, написанные ранее, обеспечивают покрытие решений и условий.

Таблица 9: Покрытие решений и условий

N	arr	substr	Ож. рез.	Факт. рез.	C1	C2	C3	C4	C5	C6	Статус
1	{"hello "world"hello"}	"hello"	2	2	F	F	F	F	F	T	Пройден
2	{"cat "dog"}	"xyz"	0	0	F	F	F	F	F	F	Пройден
3	{}	"he"	Ошибка: массив пустой	Ошибка: массив пустой	T	–	–	–	–	–	Пройден
4	{"hello"}		Ошибка: пустая подстрока	Ошибка: пустая подстрока	F	T	–	–	–	–	Пройден
5	arr из 101 эл.	"a"	Ошибка: массив слишком большой	Ошибка: массив слишком большой	F	F	T	–	–	–	Пройден
6	{"hello"}	substr дл. 1001	Ошибка: подстрока слишком длинная	Ошибка: подстрока слишком длинная	F	F	F	T	–	–	Пройден
7	{"hello str дл. 1001}	"he"	Ошибка: строка слишком длинная	Ошибка: строка слишком длинная	F	F	F	F	T	–	Пройден

Все решения и условия покрыты.

4.2.5 Комбинаторное покрытие условий

Этот критерий требует создания такого набора тестов, при котором каждая возможная комбинация результатов вычисления условий в каждом решении и каждая точка входа проверяются по крайней мере один раз.

Для условия C1 (`len(arr) == 0`) и C2 (`len(substr) == 0`) возможны следующие комбинации: (F,F), (T,–), (F,T). Условие C2 не вычисляется при C1=T, поэтому комбинация (T,T) недостижима.

Таблица 10: Комбинаторное покрытие условий

N	arr	substr	Ожид. рез.	Факт. рез.	C1	C2	Статус
1	{"hello "world "hello"}	"he"	2	2	F	F	Пройден
2	{}	"he"	Ошибка: массив пустой	Ошибка: массив пустой	T	–	Пройден
3	{"hello"}		Ошибка: пустая подстрока	Ошибка: пустая подстрока	F	T	Пройден
4	arr из 101 эл.	"a"	Ошибка: массив слишком большой	Ошибка: массив слишком большой	F	F (C3=T)	Пройден
5	{"hello"}	substr дл. 1001	Ошибка: подстрока слишком длинная	Ошибка: подстрока слишком длинная	F	F (C4=T)	Пройден
6	{"hello str дл. 1001}	"he"	Ошибка: строка слишком длинная	Ошибка: строка слишком длинная	F	F (C5=T)	Пройден
7	{"cat "dog"}	"xyz"	0	0	F	F (C6=F)	Пройден

Все достижимые комбинации условий покрыты. Комбинация C1=T и C2=T недостижима, так как при истинности C1 программа завершается до проверки C2.

4.2.6 Результаты тестирования методом белого ящика

В ходе тестирования алгоритма подсчёта вхождений подстроки в массиве строк были разработаны тесты для различных критериев покрытия кода.

Для покрытия операторов был составлен один тест с входными данными `arr = {"hello "world "hello"}` и `substr = "he"`. Ожидаемый результат `count = 2` совпал с фактическим, тест пройден. Этот тест выполнил все операторы программы, включая инициализацию счётчика, цикл `foreach` и проверку вхождения подстроки, однако не проверил ветви исключений.

Для покрытия решений потребовалось семь тестов. Первый тест с `arr = {"hello "world "hello"}` и `substr = "he"` дал результат `count = 2`, проверены условия C1=ложь, C2=ложь, C3=ложь, C4=ложь, C5=ложь, C6=истина. Второй тест с `arr = {"cat "dog"}` и `substr = "xyz"` дал результат `count = 0`, проверено условие C6=ложь и выход из цикла без совпадений. Третий тест с пустым массивом `arr = {}` проверил C1=истина и выдал сообщение об ошибке «Массив пустой». Четвёртый тест с `substr =` проверил C2=истина и выдал ошибку «Подстрока не может быть пустой». Пятый тест с массивом из 101 элемента проверил C3=истина и выдал ошибку «Длина массива слишком большая». Шестой тест с подстрокой длиной 1001 символ проверил C4=истина и выдал ошибку «Длина подстроки слишком большая». Седьмой тест с массивом, содержащим строку длиной 1001 символ, проверил C5=истина и выдал ошибку «Дли-

на строки слишком большая». Все тесты пройдены.

Для покрытия условий протестированы элементарные условия C1 и C2. Тест с корректными входными данными проверил комбинацию C1=ложь, C2=ложь. Тест с пустым массивом проверил C1=истина. Тест с пустой подстрокой проверил C2=истина. Все тесты пройдены, программа выдала ожидаемые результаты.

Для покрытия решений и условий использованы все вышеперечисленные тесты. Каждое условие C1–C6 было проверено как на истинное, так и на ложное значение, а каждый возможный исход каждого решения покрыт хотя бы одним тестом. Все семь тестов пройдены.

Для комбинаторного покрытия условий проверены все достижимые комбинации для условий C1 и C2: (ложь, ложь) с корректными входными данными, (истина, –) с пустым массивом, (ложь, истина) с пустой подстрокой. Комбинация (истина, истина) недостижима, так как при C1=истина программа завершается до вычисления C2. Дополнительно покрыты все ветви условий C3–C6. Все тесты пройдены.

5 Заключение

В ходе выполнения данной лабораторной работы были изучены методы тестирования «белым ящиком» и «чёрным ящиком».

В результате тестирования программы №1 (проверка простоты числа) методом «чёрного ящика» было составлено 30 тестов: 12 тестов на основе разбиения на классы эквивалентности, 8 тестов на основе анализа граничных условий и 6 тестов на основе причинно-следственной диаграммы. Все 30 тестов были успешно пройдены, дефектов в программе обнаружено не было. Тестирование подтвердило корректность обработки всех классов входных данных, правильность поведения на граничных значениях и соответствие всех ветвей алгоритма спецификации.

В результате тестирования программы №2 (подсчёт вхождений подстроки в массиве строк) методом «белого ящика» были разработаны тесты для пяти критериев покрытия кода. Для покрытия операторов оказалось достаточно одного теста. Для покрытия решений потребовалось семь тестов, охвативших все ветви условий C1–C6. Для покрытия условий использовались три теста, проверяющие все элементарные условия C1 и C2 на оба логических значения. Покрытие решений и условий обеспечили те же семь тестов. Для комбинаторного покрытия условий были проверены все достижимые комбинации значений C1 и C2; комбинация (истина, истина) была признана недостижимой, поскольку при C1=истина программа завершается до вычисления C2. Все тесты были пройдены успешно, ошибок в логике программы обнаружено не было.

По итогу выполненной работы можно сделать вывод, что оба метода тестирования могут быть полезны для выявления ошибок в программе, однако каждый из них ориентирован на разные аспекты качества. Метод «белого ящика» позволяет убедиться в корректности внутренней логики программы и полноте охвата всех ветвей кода, тогда как метод «чёрного ящика» проверяет соответствие поведения программы требованиям спецификации без учёта внутренней реализации. Наиболее эффективным решением является совместное применение обоих методов, что обеспечивает высокое качество программного продукта и снижает вероятность пропуска дефектов.

Список литературы

[1] Майерс Г. «Искусство тестирования программ», СПб, Диалектика, 2012г.